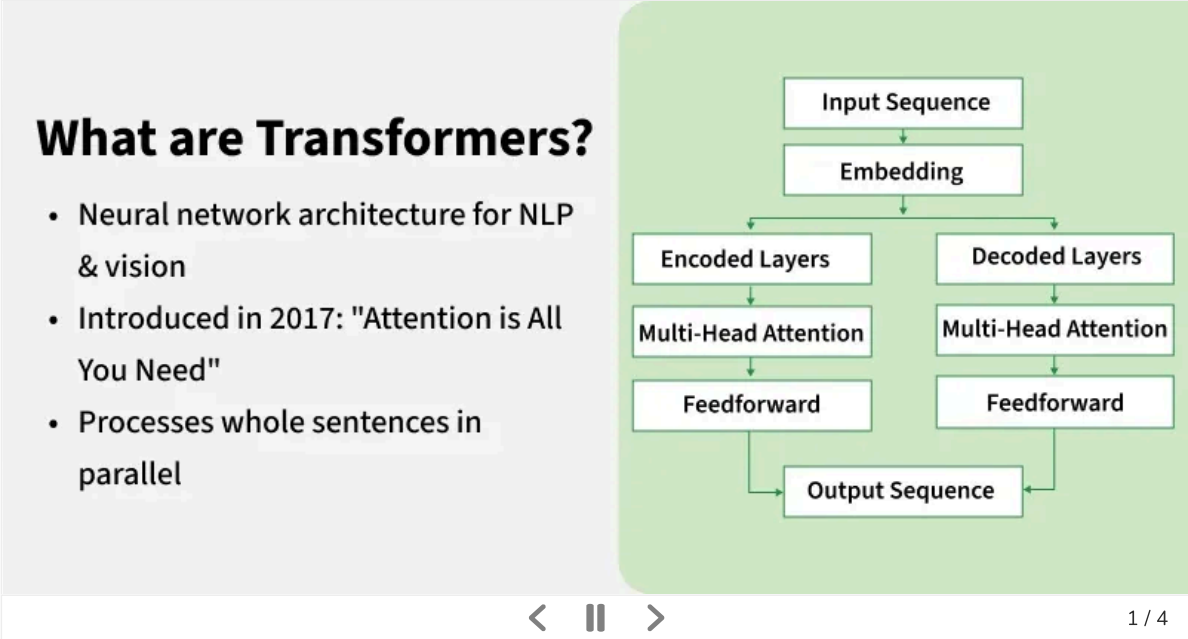
Transformers in Machine Learning

Last Updated : 11 May, 2026

Transformer is a neural network architecture used for various machine learning tasks, especially in natural language processing and computer vision. It focuses on understanding relationships within data to process information more effectively.

- Uses attention mechanisms to capture relationships between inputs
- Processes entire sequences at once instead of step by step
- Improves performance on tasks involving context and dependencies
- Widely used across NLP, vision and other AI applications



Need For Transformers Model

Transformer architecture uses attention to process an entire sentence at once instead of reading words sequentially. This helps overcome limitations of models like RNNs and LSTMs that process data step by step.

- Traditional models like RNNs (Recurrent Neural Networks) suffer from the vanishing gradient problem which leads to long-term memory loss.
- RNNs process text sequentially meaning they analyze words one at a time.

For example:

> *In the sentence: "XYZ went to France in 2019 when there were no cases of COVID and there he met the president of that country" the word "that country" refers to "France".*
>
> *However RNNs may struggle to capture long-range dependencies effectively, especially in long sequences, which can make linking distant words more difficult.*

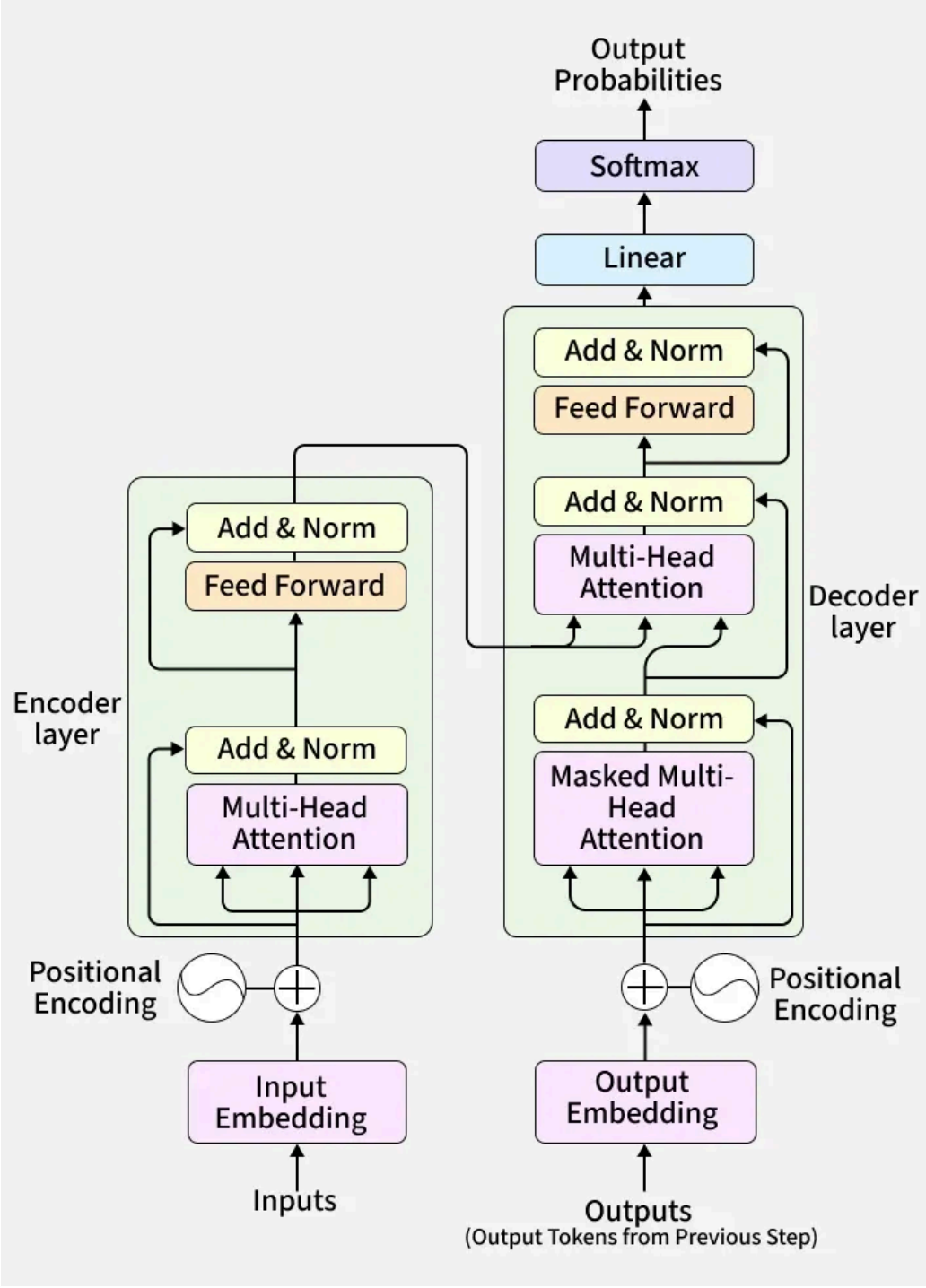
While adding more memory cells in LSTMs (Long Short-Term Memory networks) helped address the vanishing gradient issue they still process words one by one. This sequential processing means LSTMs can't analyze an entire sentence at once.

- Traditional Seq2Seq models compress the entire input into a single fixed-size context vector.
- This creates an **information bottleneck**, especially for long sequences.
- Important context may be lost during compression, reducing performance.
- Transformers solve this by using attention to access all tokens directly without compression.

For example:

> *The word "point" has different meanings in these two sentences:*
>
> - *"The needle has a sharp point." (Point = Tip)*
> - *"It is not polite to point at people." (Point = Gesture)*

Core Concepts of Transformers



Architecture of Transformer

1. Self Attention Mechanism

The self attention mechanism allows transformers to determine which words in a sentence are most relevant to each other. This is done using a scaled dot-product attention approach

Each word in a sequence is mapped to three vectors:

- Query (Q)
- Key (K)
- Value (V)

Attention scores are computed as: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

These scores determine how much attention each word should pay to others.

2. Multi-Head Attention

Instead of one attention mechanism, transformers use multiple attention heads running in parallel. Each head captures different relationships or patterns in the data, enriching the model's understanding.

3. Positional Encoding

Unlike RNNs, transformers lack an inherent understanding of word order since they process data in parallel. To solve this problem Positional Encodings are added to token embeddings providing information about the position of each token within a sequence.

4. Position-wise Feed-Forward Networks

The Feed-Forward Networks consist of two linear transformations with a ReLU activation. It is applied independently to each position in the sequence.This transformation helps refine the encoded representation at each position.

Mathematically:

$$FFN(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

5. Add & Norm (Residual Connections and Layer Normalization)

Each sub-layer in the transformer (such as self-attention and feed-forward networks) is followed by an Add & Norm step.

- **Residual Connection (Add):** The input of a layer is added to its output. This helps in preserving information and prevents vanishing gradients in deep networks.
- **Layer Normalization (Norm):** The combined output is normalized to stabilize training and improve convergence.

This structure ensures better gradient flow and allows training of deep transformer models efficiently.

6. Embeddings

Transformers cannot work with raw words as they need numbers. So, each input token (word or subword) is converted into a vector, called an embedding.

- Both encoder input tokens and decoder input tokens are converted into embeddings.

- These embeddings are trainable, meaning the model learns the best numeric representation for each token.
- The same weight matrix is shared for Encoder embeddings, Decoder embeddings and the final linear layer before softmax
- The embeddings are scaled by model to keep values stable before adding positional encoding.

7. Encoder-Decoder Architecture

The **encoder-decoder** structure is key to transformer models. The encoder processes the input sequence into a vector, while the decoder converts this vector back into a sequence. Each encoder and decoder layer includes self-attention and feed-forward layers.

> *For example, a French sentence "Je suis étudiant" is translated into "I am a student" in English.*

Transformers apply attention in three different places:

1. Encoder Self-Attention

- Q, K, V all come from the encoder's previous layer.
- Every word can attend to every other word in the input.
- This helps the encoder understand full context (long-range meaning).

2. Decoder Self-Attention (Masked)

- Q, K, V all come from the decoder's previous layer.
- Future tokens are masked (blocked), so each position only sees previous tokens.
- This keeps decoding auto-regressive i.e the model predicts one word at a time.

3. Encoder–Decoder Attention

- Queries come from the decoder.
- Keys and Values come from the encoder output.
- This lets the decoder look at important parts of the input sentence while generating output.

Together, these three attention types allow the transformer to read the entire input at once and then generate outputs step-by-step with full context.

8. Softmax Layer for Output Prediction

After the decoder processes the sequence, it must predict the next token.

- The decoder output is passed through a linear layer (whose weights are shared with embeddings).
- Then the softmax function converts these scores into probabilities.
- The token with the highest probability becomes the predicted next word.

Applications

- **NLP Tasks**: Transformers are used for machine translation, text summarization, named entity recognition and sentiment analysis.
- **Speech Recognition**: They process audio signals to convert speech into transcribed text.
- **Computer Vision**: Transformers are applied to image classification, object detection and image generation.
- **Recommendation Systems**: They provide personalized recommendations based on user preferences.
- **Text and Music Generation**: Transformers are used for generating text like articles and composing music.

Limitations

- Require high computational power, making them expensive to train
- Consume large memory, especially for long sequences
- Need large datasets to perform well
- Have input length limits, making very long text handling difficult
- Can behave like a black box, reducing interpretability
- Risk of overfitting if not properly regularized



Transformers in Machine Learning



Transformers Explained | Natural Language Processing (NLP)

Transformers in Machine Learning

Visit Course

Suggested Quiz 5 Questions

What is a key advantage of Transformers over RNNs?

A They process tokens in a fixed sequence

B They require fewer training examples

C They process entire sequences in parallel

D They remove the need for embeddings

Login to View Explanation 1/5 < Previous Next >

Comment

E error_... 33

Explore

Machine Learning Basics

Python for Machine Learning

Feature Engineering

Supervised Learning

Unsupervised Learning

Model Evaluation and Tuning

Advanced Techniques

Machine Learning Practice

Courses

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GET IT ON Google Play
Download on the App Store

Company
About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Training Program

Explore
POTD
Job-A-Thon
Blogs
Nation Skill Up

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos
DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved